

TruthLens — A Multi-Agent AI System for Detecting, Tracing and Countering Online Misinformation

An Agentic AI Ecosystem Against Misinformation

Institution: M.H. Saboo Siddik College of Engineering | **Track:** Engg-Spoke | **Domain:** GenAI | **Duration:** 10 Days

1. Title

Working Project Title TruthLens — A Multi-Agent AI System for Detecting, Tracing and Countering Online Misinformation

Team Details

Details of all four team members. The first student is the Team Lead.

Student Name	College Name	Branch	Mobile No.	Mail ID	Role
Hajra Khan	M.H. Saboo Siddik College of Engineering	CSE (AI&ML)	+91 877 964 4270	hajra.252764.cs@mhssce.ac.in	Lead
Insha Ansari	M.H. Saboo Siddik College of Engineering	CSE (AI&ML)	+91 8655422852	insha.252766.cs@mhssce.ac.in	Member
Shifa Shaikh	M.H. Saboo Siddik College of Engineering	CSE (AI&ML)	+91 77386 30834	shifa.252768.cs@mhssce.ac.in	Member
Bushra Kazi	M.H. Saboo Siddik College of Engineering	CSE (AI&ML)	+91 91687 86455	bushra.241813.ci@mhssce.ac.in	Member

Program Track and Domain

Program Track	Domain
Engg-Spoke	GenAI (with supporting Machine Learning / Deep Learning components)

Guide Details

Guide Name Mr. Suraj Chopade
Designation Senior Trainer

2. Problem Statement

Background

Online misinformation now travels faster than the institutions built to correct it. A single misleading claim about a disease outbreak, an election procedure, a natural disaster or a financial market can be republished across news portals, video platforms and messaging apps within minutes. Professional fact-checking, by contrast, is a manual, human-paced process: an analyst must first notice the claim, then search for whether anyone has already debunked it, then trace where it started, then judge how far it is likely to travel, and only then write and publish a correction. By the time that correction exists, the claim has usually already reached the audience it was going to reach.

Why we chose this problem

Every member of this team has personally received forwarded claims that turned out to be false, and has watched how difficult it is for an ordinary reader to check one quickly. We were also interested in agentic AI — systems in which several specialised AI components cooperate on stages of a single task rather than one model attempting everything. Misinformation response is a naturally staged problem (detect → trace → predict → counter), which makes it an honest fit for a multi-agent architecture rather than a forced one.

Why it matters in real life

False health claims discourage treatment, false civic claims suppress or distort participation, false disaster claims send people towards danger, and false financial claims trigger panic. The harm in each case is caused less by the existence of the claim than by the delay before a credible correction reaches the same audience. Reducing that delay is therefore the highest-leverage intervention available.

The exact problem or gap

Existing tools address fragments of the problem. Classifiers label a piece of text as true or false but say nothing about where it came from or who will see it next. Social-network research models diffusion but assumes the false claim has already been identified. Fact-checking organisations produce excellent corrections but publish them hours or days later, and readers must actively go and look for them. No accessible, end-to-end system takes a live post from a real public source, decides whether it contains a harmful false claim, reconstructs its provenance, estimates its spread risk, and drafts an evidence-backed correction as one continuous, auditable pipeline. Crucially, most generative approaches to this problem also hallucinate: they will happily invent a citation to support a rebuttal, which is precisely the failure mode a misinformation tool cannot afford.

What we plan to solve

TruthLens closes that gap with four cooperating AI agents behind a single orchestrated workflow. It ingests live content from public sources, detects candidate false claims with a calibrated severity and confidence score, traces provenance through a graph database, predicts spread risk from explainable weighted features, and drafts a counter-narrative that is strictly grounded in retrieved fact-check evidence — returning an explicit `no_evidence` state rather than fabricating a citation when the knowledge base has nothing relevant. High-severity cases are routed to a human reviewer before anything is published, so the system augments a human fact-checker rather than replacing one.

3. Objectives

- 1 **Build a live ingestion layer** that continuously collects public content from real sources — NewsAPI, the YouTube Data API and RSS feeds — normalises it into a common post schema, and remains extensible to further platforms.
- 2 **Implement an autonomous Claim Detector agent** that uses a large language model to decide whether an ingested post contains a factual claim, whether that claim is likely misinformation, and to assign a calibrated severity (1–10), a confidence score, a harm category and the persuasion techniques used.
- 3 **Eliminate duplicate reporting** through semantic claim resolution, so that many paraphrases of the same underlying assertion collapse into one canonical claim rather than flooding the queue.
- 4 **Implement an Origin Tracer agent** that reconstructs the provenance of each claim as a graph of claims, sources and reports in a graph database, and produces a readable natural-language provenance summary.
- 5 **Implement a Spread Predictor agent** that estimates spread risk and a reproduction-style amplification figure from explainable weighted features, reporting the numeric contribution of every driver rather than returning an opaque score.
- 6 **Implement a Narrative Drafter agent** that composes a counter-narrative grounded in evidence retrieved from a curated fact-check corpus, and that refuses to answer when no sufficiently relevant evidence exists.
- 7 **Orchestrate all four agents** as a single stateful workflow with conditional routing, so that low-value signals are dropped early and only meaningful claims consume downstream computation.
- 8 **Keep a human in the loop** by routing high-severity, high-confidence claims into a review queue that must be cleared by a person before a counter-narrative can be published.
- 9 **Deliver a working web prototype** that presents every stage of the pipeline against live backend data, with no mock or hard-coded content anywhere in the interface.

4. Methodology / Implementation Plan

The system is organised as a five-stage pipeline. Stages 2 to 5 are implemented as four independent AI agents coordinated by a stateful orchestration graph; a shared state object is passed from agent to agent and enriched at each step, and conditional edges decide whether a given claim continues through the pipeline or is dropped.

Overall Workflow

Stage	Component	What it does
1	Ingestion Layer	Polls public sources on a schedule, normalises each item into a common post record (text, author, platform, URL, timestamp, engagement signals, a heuristic suspicion score), and repairs encoding artefacts before the text reaches any model.
2	Agent 1 — Claim Detector	Sends the normalised post to a large language model under a structured rubric with explicit severity anchors and explicit exclusions for legitimate reporting. Returns claim text, misinformation verdict, severity, confidence, category, rationale, harm type, persuasion techniques and keywords. A routing rule then marks the claim dropped, pending or review_required.
2b	Semantic Claim Resolution	Embeds the extracted claim and compares it against an index of canonical claims. If an existing claim lies within a calibrated distance threshold the new post is attached to it; otherwise a new canonical claim is created. This is what prevents the same story, reported by twelve outlets, from becoming twelve claims.
3	Agent 2 — Origin Tracer	Writes the claim, its sources and their reporting relationships into a graph database, then queries that graph for the earliest observed report, the ordered timeline of outlets, and the breadth of independent sources. A language model converts these graph facts into a readable provenance narrative — the facts come from the graph, the prose comes from the model.
4	Agent 3 — Spread Predictor	Computes a spread-risk score from five weighted, normalised features and derives an amplification estimate from it. Every driver is reported with its own numeric contribution to the final score, so the output is inspectable rather than a black box.
5	Agent 4 — Narrative Drafter	Retrieves the most semantically relevant documents from the fact-check corpus using vector search, and asks the language model to write a correction using only those documents. If nothing passes the relevance threshold, the agent returns no_evidence instead of a fabricated rebuttal.
6	Human Review & Publication	Claims above the severity and confidence gate wait in a review queue. A reviewer approves or rejects; the approved counter-narrative can then be exported, copied, or dispatched through a simulated publication step. Every decision is recorded as an audit event in the graph.

Algorithms, Models and Techniques

Technique	Applied in	Purpose
Large Language Model — openai/gpt-oss-20b served via Groq	Agents 1, 2 and 4	Claim extraction and scoring, provenance summarisation, grounded counter-narrative generation.
Structured prompting with severity anchors and negative exclusions	Agent 1	Calibrates the model so that ordinary factual reporting is not escalated as misinformation.
Sentence-transformer embeddings — all-MiniLM-L6-v2	Claim resolution and evidence retrieval	Maps text into a vector space so that semantic similarity can be measured numerically.
Vector similarity search (squared L2 distance) with empirically calibrated thresholds	Claim resolution, Agent 4	Deduplicates paraphrased claims and selects relevant evidence documents.
Graph traversal over a labelled property graph	Agent 2	Recovers first-seen source, propagation timeline and outlet breadth from recorded relationships.
Weighted multi-feature scoring model with penalty term	Agent 3	Produces an explainable risk score whose drivers can each be attributed a share of the total.

Technique	Applied in	Purpose
Graph neural network library (PyTorch Geometric)	Agent 3	Supports graph-structured representation of the propagation network for spread estimation.
Retrieval-Augmented Generation with a strict grounding constraint	Agent 4	Forces every counter-narrative to cite retrieved evidence, and forbids answering without it.
Stateful agent orchestration with conditional edges	Whole pipeline	Routes each claim through only the stages it needs, and terminates low-value signals early.
Rule-based domain classification into a seven-domain taxonomy	Claims and evidence	Gives claims and evidence a shared vocabulary so they can be grouped and cross-referenced coherently.

Spread-Risk Feature Weights (Agent 3)

Feature	Weight	Source of the value
Detector severity	0.30	Agent 1 output
Ingest suspicion score	0.20	Ingestion heuristics
Spread velocity	0.20	Provenance timeline
Outlet count	0.15	Provenance graph
Detector confidence	0.15	Agent 1 output

Each driver is reported to the user in the form “Detector severity: 7/10 → +0.210 of 0.396 risk (53%)”, so the contribution of every feature to the final score is visible rather than inferred.

Tools and Technologies

Layer	Technologies
Language & Runtime	Python (backend and agents), TypeScript / Node.js (frontend and adapter layer)
Agent Orchestration	LangGraph — a stateful graph of agent nodes with conditional edges
LLM Inference	Groq API serving openai/gpt-oss-20b, accessed through a shared client with rate-limit backoff
Vector Database	ChromaDB — two collections, one for fact-check knowledge and one for the canonical claim index
Graph Database	Neo4j Aura — nodes for Claim, Source, Review and AuditEvent
Embeddings	Sentence-Transformers all-MiniLM-L6-v2
Deep Learning	PyTorch, PyTorch Geometric
API Layer	FastAPI with CORS, exposing claim, evidence, source and publication endpoints
Adapter Layer	Express server acting as a backend-for-frontend, with Vite middleware in development
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4, Framer Motion, lucide-react
Data Sources	NewsAPI, YouTube Data API, RSS feeds; connectors scaffolded for Reddit (PRAW), Telegram (Telethon), Twitter/X and the TikTok Research API
Development Tools	VS Code, Git and GitHub, Node.js / npm, Python virtual environments

5. Resources and Timeline

A. Hardware & Software Requirements

Hardware

Component	Specification
Processor	Intel Core i5 (13th Generation) or equivalent
RAM	16 GB
Storage	512 GB SSD
Graphics	Intel Iris Xe Graphics
Operating System	Windows 11 Pro (64-bit)

No dedicated GPU is required. Large-language-model inference is performed remotely through the Groq API, and the embedding model used for retrieval is small enough to run comfortably on CPU.

Software

Category	Items
Programming Languages	Python, TypeScript / JavaScript
Backend Frameworks	FastAPI, LangGraph, Uvicorn
Frontend Frameworks	React 19, Vite, Tailwind CSS v4, Framer Motion
Databases	ChromaDB (vector), Neo4j Aura (graph, cloud-hosted)
ML / AI Libraries	Sentence-Transformers, PyTorch, PyTorch Geometric, Groq Python SDK
External APIs	Groq API, NewsAPI, YouTube Data API v3
Editors & Version Control	Visual Studio Code, Git, GitHub
Runtime	Node.js with npm, Python 3 with a virtual environment

B. Dataset

The project does not use a single static downloadable dataset. Because TruthLens is designed to operate on live content, its data is acquired at run time from two distinct streams.

1. Live content stream (the input to be analysed)

- **NewsAPI** — news articles and headlines matching the monitored topic set (<https://newsapi.org>).
- **YouTube Data API v3** — video metadata, titles and descriptions for monitored topics.
- **RSS feeds** — direct syndication from news and fact-checking publishers.
- Connectors have additionally been scaffolded for Reddit, Telegram, Twitter/X and TikTok so that further sources can be enabled once the corresponding API credentials are obtained.

2. Fact-check evidence corpus (the knowledge the system reasons against)

This corpus is built by the team rather than downloaded. A seeding routine harvests published fact-checks from the public RSS feeds of established fact-checking organisations — **Snopes**, **PolitiFact**, **FactCheck.org**, **Full Fact** and the **World Health Organization** — and supplements them with topic-matched articles retrieved for each monitored subject area. Each document is embedded and stored in the vector database with its domain label and source attribution. The corpus currently holds **123 documents**, grown from an initial three, and it expands automatically as new fact-checks are published.

Why this design was chosen. A static misinformation dataset would be stale on the day it was downloaded, and a model trained on it would be unable to reason about claims that emerged afterwards. Retrieving evidence at query time from a corpus that continuously refreshes lets the system respond to claims it has never seen before, and makes every correction traceable to a named, dated, external source.

C. Project Timeline

The project was executed over a ten-day schedule.

Phase	Task	Description	Tentative Completion
Phase 1	Requirement Analysis	Understand the problem, define objectives, identify requirements and project scope.	Day 1
Phase 2	Research & Planning	Research existing solutions, select technologies, and finalise the proposed approach.	Day 2
Phase 3	System Design	Design system architecture, workflow, modules, database/API structure, and UI plan.	Day 3
Phase 4	Development — Phase 1	Set up the development environment and implement the basic project structure and backend.	Day 4
Phase 5	Development — Phase 2	Implement the core functionality and major modules.	Day 5
Phase 6	AI / ML Integration	Integrate AI/ML models, APIs, algorithms and intelligent components used in the project.	Day 6
Phase 7	Frontend & Integration	Develop and refine the user interface and connect frontend, backend and AI components.	Day 7
Phase 8	Testing & Debugging	Test individual modules, identify errors, fix bugs, and improve system reliability.	Day 8
Phase 9	Optimisation & Finalisation	Optimise performance, improve UI/UX, verify all features, and conduct final testing.	Day 9
Phase 10	Documentation & Presentation	Prepare the synopsis and report, screenshots, diagrams, presentation and final demonstration.	Day 10

6. Expected Outcomes / Conclusion

Deliverables

- 1 **A working end-to-end prototype** — a web application in which a live post travels through all four agents and emerges as a scored, traced, risk-rated claim with a drafted counter-narrative.
- 2 **Four functioning AI agents** — Claim Detector, Origin Tracer, Spread Predictor and Narrative Drafter — coordinated by a single orchestration graph.
- 3 **A self-maintaining fact-check knowledge base** that grows automatically from public fact-checking feeds.
- 4 **A provenance graph** recording claims, sources, reporting relationships, human reviews and audit events.
- 5 **A human-in-the-loop review workflow** with export, copy and simulated publication of approved counter-narratives.
- 6 **A fully live interface** in which every figure displayed is read from the backend; all mock data was removed from the codebase.

Results Observed During Development

Metric	Result	Comment
End-to-end pipeline latency	3.0 s average <i>range 0.9 – 7.5 s</i>	Measured across the full four-agent pipeline. The original design target was under 90 seconds; the achieved latency is substantially below it.
Human review queue load	53% → 15%	Proportion of claims escalated for human review, before and after calibrating the detector rubric and the escalation gate.
Semantic deduplication accuracy	8 / 8	Correct decisions on the calibration set. True paraphrases measured 0.13–0.58 distance; genuinely distinct claims measured above 0.80, giving a clean separation at the 0.70 threshold.
Claim consolidation	20 posts → 16 claims	Duplicate reporting of the same assertion collapsed into single canonical claims.
Evidence corpus growth	3 → 123 documents	After introducing topic-matched seeding from fact-check RSS feeds.
Evidence linkage	6 documents	Retrieved evidence documents successfully bound to specific active claims.

Key Technical Findings

- **Retrieval threshold calibration mattered more than model choice.** The counter-narrative agent initially returned `no_evidence` for every claim. The cause was a retrieval distance threshold set too tightly at 0.6, which admitted only near-verbatim matches. A claim about an asteroid was correctly matched to a NASA document at distance 0.822 — a genuinely correct match that the threshold was rejecting. Raising the threshold to 1.2 restored retrieval without introducing false matches.
- **Prompt calibration is a measurable engineering activity.** Adding explicit severity anchors and explicit exclusions for legitimate reporting reduced the human review burden from 53% of claims to 15%, without any change of model.
- **Deduplication is a prerequisite for provenance, not a nicety.** Until paraphrases of the same claim were collapsed, the Origin Tracer saw one source per claim and had no propagation path to reconstruct, which in turn starved the Spread Predictor of its outlet-count and velocity features.
- **Refusing to answer is a feature.** Requiring the drafter to return `no_evidence` rather than generate an unsupported rebuttal is what makes the system’s output trustworthy in a domain where a fabricated citation would be worse than silence.

Limitations

- Spread prediction uses an explainable weighted-feature model rather than a model trained on historical propagation data; its output is a calibrated heuristic, not a statistically validated forecast.

- Provenance reconstruction is limited to sources the system has actually observed. It records first-seen order within the ingested set, which is not necessarily true origin.
- Coverage depends on the ingestion sources currently enabled. Closed messaging platforms, where a great deal of misinformation actually circulates, are not accessible through public APIs.
- Counter-narrative quality is bounded by the coverage of the evidence corpus; a novel claim with no published fact-check will correctly but unhelpfully return `no_evidence`.
- Publication is simulated. No content is transmitted to any external platform or authority.
- Evaluation was conducted on a modest volume of live data over the project period and not against a large, independently labelled benchmark.

Future Scope

- Enable the Reddit, Telegram, Twitter/X and TikTok connectors once API access is granted, widening coverage to the platforms where claims spread fastest.
- Train the spread model on observed propagation histories collected by the system itself, converting the heuristic into a learned predictor.
- Add multilingual ingestion and detection, which is essential for the Indian information environment.
- Add image and video claim analysis, since a growing share of misinformation is not textual.
- Build an integration path to a recognised fact-checking body so approved corrections can be genuinely published rather than simulated.

Conclusion

TruthLens demonstrates that the misinformation response cycle — detect, trace, predict, counter — can be compressed from hours into seconds by decomposing it into specialised cooperating AI agents rather than asking a single model to do everything. The project’s central engineering claim is not that a language model can label a claim as false; that is now routine. It is that a generative system can be constrained to be *useful and honest* in a domain where confident invention is the primary risk: every counter-narrative is bound to retrieved external evidence, every risk score exposes the numeric contribution of each of its drivers, every provenance statement derives from recorded graph relationships, and every high-severity case is held for human judgement before anything is published. The measured average pipeline latency of 3.0 seconds against a 90-second design target indicates that real-time operation at useful scale is achievable, while the reduction of human review load from 53% to 15% shows that such a system can be made to respect, rather than overwhelm, the scarce human attention it is designed to support.

7. References

Framework and Library Documentation

- 1 LangGraph Documentation — LangChain. <https://langchain-ai.github.io/langgraph/>
- 2 Groq API Documentation — Groq. <https://console.groq.com/docs>
- 3 Chroma Documentation — Chroma. <https://docs.trychroma.com/>
- 4 Neo4j Aura and Cypher Documentation — Neo4j. <https://neo4j.com/docs/>
- 5 Sentence-Transformers Documentation, including the all-MiniLM-L6-v2 model card. <https://www.sbert.net/>
- 6 PyTorch Geometric Documentation. <https://pytorch-geometric.readthedocs.io/>
- 7 FastAPI Documentation. <https://fastapi.tiangolo.com/>
- 8 React Documentation. <https://react.dev/>
- 9 Tailwind CSS v4 Documentation. <https://tailwindcss.com/docs>

Data Source Documentation

- 10 NewsAPI Documentation. <https://newsapi.org/docs>
- 11 YouTube Data API v3 Documentation — Google Developers. <https://developers.google.com/youtube/v3>

Fact-Check Sources Used to Build the Evidence Corpus

- 12 Snopes. <https://www.snopes.com/>
- 13 PolitiFact. <https://www.politifact.com/>
- 14 FactCheck.org. <https://www.factcheck.org/>
- 15 Full Fact. <https://fullfact.org/>
- 16 World Health Organization. <https://www.who.int/>

Project Repository

- 17 TruthLens source repository — GitHub. <https://github.com/Insha201/TruthLens>